

1. Cum contribuie Tag Helper-ul asp-validation-for la funcționalitatea formularului în exemplul de mai jos?

```
<form asp-action="Register" asp-controller="Account">  
  <input asp-for="Email" type="email" class="form-control" />  
  <span asp-validation-for="Email" class="text-danger"></span>  
  <button type="submit">Register</button>  
</form>
```

- ☐ asp-validation-for integrează automat o funcționalitate de autocomplete pentru câmpul Email, sugerând adrese de email pe baza celor mai frecvent utilizate de către alți utilizatori ai aplicației;
- ☐ asp-validation-for automatizează criptarea datelor din câmpul Email pentru a asigura securitatea informațiilor transmise;
- ☐ asp-validation-for afișează mesaje de eroare de validare generate pe server pentru proprietatea specificată, în acest caz Email, îmbunătățind feedback-ul utilizatorului în cazul datelor introduse invalide;
- ☐ Tag Helper-ul asp-validation-for este utilizat pentru a optimiza performanța trimiterii formularului, prin pre-procesarea datelor pe client înainte de a fi trimise la server;

2. Ce rol joacă un DbSet într-un DbContext, conform descrierii următoare?

```
public class SchoolContext : DbContext
{
    public DbSet<Student> Students { get; set; }
    public DbSet<Teacher> Teachers { get; set; }
}
```

- ☐ Proprietățile DbSet sunt utilizate pentru a defini relațiile dintre entități (1:1; 1:m; m:m);
- ☐ DbSet controlează versiunea bazei de date și aplică automat upgrade-uri și downgrade-uri la schema bazei de date conform versiunilor de software ale aplicației;
- ☐ Un DbSet este utilizat pentru a configura setările specifice pentru operațiunile de query și save asupra entităților pe care le gestionează, oferind o interfață directă către tabelul corespunzător din baza de date;
- ☐ Proprietățile DbSet sunt utilizate pentru a configura baza de date și nu sunt implicate în operațiunile de interogare sau actualizare a datelor;

3. În arhitectura Model-View-Controller (MVC) utilizată în ASP.NET Core 8.0, ce reprezintă "Modelul" și care este contribuția sa principală la funcționarea aplicației?

- ☐ Modelul reprezintă un punct central pentru logica aplicației și validarea datelor, fiind responsabil cu gestionarea datelor din aplicație și manipularea acestora;
 - ☐ Modelul gestionează toate datele aplicației, primindu-le direct din componenta View, asigurând astfel o conexiune dinamică între datele utilizatorului și logica aplicației;
 - ☐ Modelul se ocupă de gestionarea, procesarea și manipularea datelor în cadrul aplicației, interacționând predominant cu Controller-ul pentru a asigura fluxul corect al informațiilor;
 - ☐ Modelul funcționează ca un strat fundamental în procesarea și manipularea datelor, facilitând comunicarea eficientă între Controller și View prin intermediul datelor procesate;
-

4. Care este scopul principal al fișierului Program.cs într-o aplicație .NET?

- ☐ Program.cs este folosit pentru a gestiona conexiunile la baza de date și pentru a executa interogări SQL, facilitând operațiile de CRUD în aplicație;
- ☐ Program.cs este esențial pentru configurarea serviciilor necesare aplicației, inclusiv middleware și servicii de dependență, facilitând astfel o inițializare corespunzătoare și configurare detaliată a aplicației;
- ☐ Fișierul Program.cs este folosit pentru configurarea serviciilor externe de monitorizare și analiză, implementând direct în cod instrumente pentru tracking-ul performanței și gestionarea erorilor în aplicație;
- ☐ Program.cs este utilizat pentru a defini și stoca setările de configurare ale aplicației, inclusiv detalii legate de medii diferite de dezvoltare și producție;

5. Care dintre următoarele afirmații descrie corect funcționalitatea Model Binding-ului în ASP.NET MVC Core?

- ☐ În ASP.NET MVC, Model Binding-ul reduce nevoia de codare manuală pentru extragerea datelor din cereri, utilizând convenții și configurări predefinite pentru a popula obiectele Modelului direct din datele trimise de utilizator prin View;
- ☐ Model Binding-ul permite executarea directă a scripturilor de pe client pentru validarea datelor înainte ca acestea să fie trimise la server, integrând automat validarea JavaScript în fiecare formular;
- ☐ Model Binding-ul este responsabil pentru configurarea și administrarea sesiunilor utilizatorilor în aplicații, menținând starea între diferite cereri HTTP;
- ☐ Model Binding-ul controlează fluxul de navigație într-o aplicație ASP.NET, determinând ce pagină sau View trebuie să fie afișată în urma unei acțiuni din Controller;

6. Cum se implementează autentificarea și autorizarea folosind sistemul Identity într-o aplicație ASP.NET Core?

- ☐ Autorizarea în ASP.NET Core cu sistemul Identity este gestionată exclusiv prin verificări manuale efectuate de administratori, care trebuie să aprobe fiecare cerere de acces înainte ca utilizatorii să poată interacționa cu aplicația;
- ☐ Dezvoltatorii pot utiliza sistemul Identity pentru a gestiona autentificarea în ASP.NET Core, configurând serviciile necesare și middleware-ul în clasa Program.cs, folosind apoi componente precum SignInManager și UserManager pentru autentificarea utilizatorilor;
- ☐ În ASP.NET Core, sistemul Identity este folosit pentru a cripta traficul de rețea între client și server, asigurând securitatea comunicațiilor;
- ☐ În ASP.NET Core, sistemul Identity este configurat prin adăugarea unui plugin de browser care gestionează autentificarea utilizatorilor și autorizarea acestora direct de pe partea client, fără interacțiuni cu serverul;

7. Analizați următoarele afirmații despre constructorul ApplicationDbContext în contextul utilizării cu Entity Framework. Ce afirmație descrie cel mai bine scopul și funcționalitatea acestuia?

```
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }
    ...
}
```

- ☐ Constructorul ApplicationDbContext automatizează crearea schemelor de bază de date și tabelurilor asociate entităților modelate în context, activând un script de migrație la fiecare instanțiere;
- ☐ Acest constructor este considerat un standard obligatoriu pentru toate clasele care extind DbContext, fiind utilizat exclusiv pentru a îndeplini cerințele de compilare fără a influența configurarea sau inițializarea contextului bazei de date;
- ☐ Constructorul ApplicationDbContext permite injectarea dependențelor necesare pentru configurarea contextului, inclusiv specificarea opțiunilor precum șirul de conectare la baza de date și alte setări particularizate ale acesteia, facilitând astfel o configurare detaliată a accesului la date;
- ☐ În această implementare, constructorul are rolul de a suprascrie orice configurare preexistentă a DbContext-ului cu setări prestabilite care favorizează performanța, ignorând opțiunile transmise prin parametru, ceea ce simplifică managementul resurselor de sistem;

8. Cum mapează ASP.NET MVC Core rutele la metodele corespunzătoare din Controllere?

- ☐ În ASP.NET MVC, rutele sunt gestionate printr-un router central, care analizează URL-urile cererilor și le mapează la acțiunile Controller-elor bazate pe cele mai bune potriviri în configurația de rutare;
 - ☐ În ASP.NET MVC, toate rutele sunt stabilite automat de framework fără posibilitatea de personalizare din partea dezvoltatorului, asigurând un standard uniform de tratare a URL-urilor;
 - ☐ ASP.NET MVC mapează toate rutele direct în IIS, utilizând un modul special care primește cererile înainte să ajungă la aplicație pentru a determina Controllerul potrivit;
 - ☐ Rutarea în ASP.NET MVC este realizată prin definirea unui singur Controller global care procesează toate cererile și apoi le distribuie metodelor, analizând conținutului cererii;
-

9. Analizați următoarele afirmații despre Partial View și Layout View și alegeți opțiunea corectă care descrie diferențele dintre cele două:

1. Partial View este proiectat pentru a fi reutilizat în diverse părți ale aplicației și poate fi încărcat dinamic, pe când Layout View definește structura de bază a paginii și este utilizat ca șablon principal pentru toate paginile, asigurând consistența designului;
2. Layout View este utilizat exclusiv pentru elemente statice ale interfeței, cum ar fi textul și imaginile, în timp ce Partial View este destinat gestionării elementelor dinamice, cum ar fi formularele și listele interactive;
3. Layout View gestionează conexiunile la baza de date și interogările necesare pentru a asambla conținutul paginilor, în timp ce Partial View manipulează doar elementele vizuale fără a accesa direct datele;
4. Partial View și Layout View sunt identice în funcționalitate și utilizare, ambele servind la structurarea vizuală a conținutului, singura diferență fiind denumirea și contextul în care sunt folosite;

- ☐ Doar 1 este corectă;
- ☐ Toate afirmațiile sunt corecte;
- ☐ 2 și 3 sunt corecte;
- ☐ 1 și 2 sunt corecte;

10. Analizați următoarele afirmații despre mecanismul de Lazy Loading și determinați care dintre ele sunt adevărate:

1. Lazy Loading încarcă datele doar când acestea sunt efectiv necesare, ceea ce poate îmbunătăți performanța inițială a aplicației prin reducerea timpului de încărcare la startup;
2. Proprietățile trebuie să fie declarate ca public virtual în cadrul claselor, astfel încât Entity Framework să poată suprascrie aceste proprietăți în clasele derivate pentru a activa mecanismul de Lazy Loading;
3. În momentul interogării, mecanismul de Lazy Loading încarcă întotdeauna toate datele asociate entității, inclusiv colecțiile și proprietățile de navigație;
4. Lazy Loading poate scădea performanța aplicației prin creșterea cantității de date încărcate inițial din baza de date;

- ☐ 1 și 3;
- ☐ Doar varianta 1;
- ☐ 1 și 2;
- ☐ 2 și 4;